

AI SHOPPING AGENT

Multimodal Conversational Assistant for Product Discovery & Safe Checkout

LangChain Tool-Calling Agent · Multimodal Vision · Groq LLMs · Streamlit Interface

4

Agent Tools

2

Input Modes

3

DB Tables

Streamlit

Deployment

Portfolio Project | Sajal Agarwal | AI & Machine Learning Engineering

AGENDA

What We Will Cover

01

Problem & Business Context

Why fragmented online shopping needs a smarter assistant

02

Solution & Core Features

Conversational, multimodal, confirmation-gated design

03

System Architecture

LangChain agent, tool layer, model layer, and data layer

04

Interaction Flows

Text-based browsing and image-based discovery journeys

05

Tech Stack & Data Model

Python, LangChain, Groq, and the SQLite schema

06

Safety Guardrails & Takeaways

Confirmation gates, portfolio value, and next steps

PROBLEM STATEMENT

Why Online Shopping Needs a Smarter Assistant

The Challenge

- Search is keyword-only and often misses the user's real buying intent
- Ratings and reviews are disconnected from the search experience
- Recommendation-to-purchase flow is rarely conversational
- Image-based product discovery is almost never integrated into one assistant

Our Solution

Conversational Intent Understanding

Interprets natural-language shopping requests, not just keywords

Tool-Calling Architecture

Dedicated tools for search, ratings, checkout, and vision parsing

Multimodal Discovery

Upload a product photo and the agent searches by image

Transaction Safety

Hard guardrails require explicit confirmation before any order

CORE FEATURES

What the Shopping Agent Can Do

Conversational Discovery

Natural-language search with optional max-price and organic filters

Rating-Aware Ranking

Aggregated review metrics merged directly into the recommendation flow

Confirmation-Gated Checkout

Orders are placed only after the user explicitly confirms

Multimodal Image Search

Upload a photo; a vision model extracts attributes to search with

Streamlit Chat UI

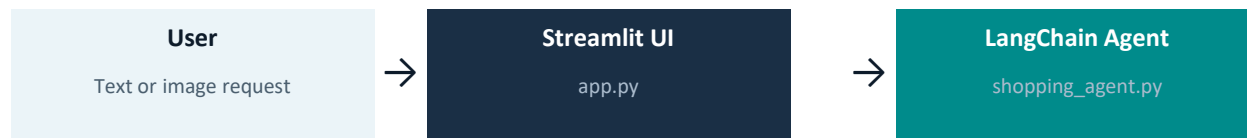
Chat-first interface with a sidebar image-upload flow

SQLite Persistence

Local relational store for products, reviews, and orders

Agent, Tools, and Data Working Together

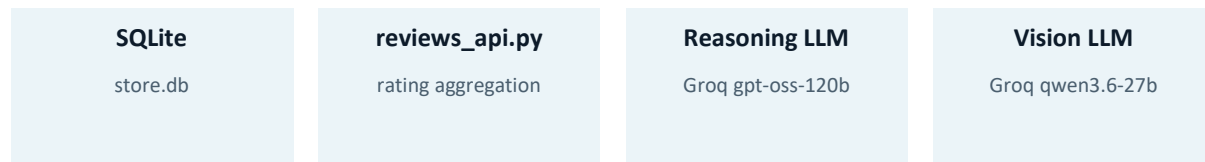
Request Pipeline



Tool Layer



Data & Model Layer



Why This Design?

- Explicit tools keep the LLM's responsibilities auditable and testable
- Separate reasoning and vision models optimize cost and latency per task
- SQLite keeps the demo deterministic, local, and reproducible
- The confirmation gate lives in the system prompt, governing every order

Four Tools Give the Agent Its Capabilities

Tool	Inputs	Data Source	Description
search_products	query, max_price, is_organic	products table	Keyword search with optional price and organic filters
get_rating	product_id	reviews_api.py → reviews table	Aggregate average rating and total review count
checkout	product_id	orders table	Creates an order — only runs after explicit user confirmation
describe_product_image	image_path	Vision LLM (Groq qwen/qwen3.6-27b)	Extracts product_type, search_query, is_organic, description as JSON

DATA MODEL: products · reviews · orders — SQLite (store.db)

From Natural-Language Request to Confirmed Order

Browsing & Checkout Flow

- 1 User Message**
"Organic honey under \$20 with rating ≥ 4.5 "
- 2 Intent Parsing**
Agent extracts query, max_price, is_organic, min_rating
- 3 search_products**
Query the products table for matching candidates
- 4 get_rating (loop)**
Fetch average rating and review count for each candidate
- 5 Ranked Recommendations**
Numbered list with (ID:X) is shown to the user
- 6 checkout**
Runs only after the user explicitly confirms a choice

Flow Highlights

Tools Used

2 — search_products, get_rating

Guardrail

Product ID always taken from prior agent output

Data & Outcome

Data source: store.db (products + reviews tables)

Output format: plain text, numbered list with (ID:X)

Outcome: order confirmation with order ID, product name, and price

Shopping by Photo Instead of Words

Image Search Flow

- 1 Image Upload**
User uploads a product photo in the Streamlit sidebar
- 2 describe_product_image**
Vision LLM (Groq qwen/qwen3.6-27b) analyzes the photo
- 3 JSON Attributes**
Returns product_type, search_query, is_organic, description
- 4 Query Construction**
Agent reuses the extracted search_query and is_organic
- 5 search_products**
Executes the same catalog search as text browsing
- 6 Standard Flow Continues**
Ratings, recommendations, and confirmation as usual

Flow Highlights

Vision Model

Groq qwen/qwen3.6-27b

Output Contract

Strict JSON — no prose, easy to parse

Sample Inputs

resources/honey.png — positive organic match

resources/oats.png — positive category match

resources/elephant.png — negative / control sample

Guardrails That Make the Agent Trustworthy

Explicit Confirmation Required

The agent never calls checkout unless the user clearly confirms (“yes,” “the first one,” etc.)

No Guessed Product IDs

Every checkout call reuses an (ID:X) the agent itself printed in an earlier recommendation

Vision Is a Query Helper, Not a Purchase Trigger

Image analysis only feeds search_products — it can never directly trigger an order

Auditable, Reproducible Actions

Rules live in the system prompt, making every order traceable to an explicit user request

Python, LangChain, and Groq End-to-End

UI LAYER	Streamlit Chat UI + image uploader	python-dotenv Environment configuration	tempfile Temporary image storage (stdlib)
ORCHESTRATION LAYER	LangChain create_agent + tool binding	langchain-core Message & prompt primitives	Tool-Calling 4 explicit @tool functions
MODEL LAYER	Groq LLM inference provider	gpt-oss-120b Shopping reasoning model	qwen3.6-27b Vision understanding model
DATA LAYER	SQLite store.db relational database	sqlite3 Stdlib database driver	reviews_api.py Rating aggregation logic

LIVE DEMO

Streamlit Chat Interface — Sample Conversation



AI Shopping Assistant

I want organic honey under \$20 with a 4.5+ rating

#1. Wild Forest Honey (ID:3) — \$14.99 ★4.7 — organic

Would you like to order it?

Yes, place the order

Order #12 confirmed! 'Wild Forest Honey' has been ordered for \$14.99.
Arrives in 3-5 business days.

Why This UX?

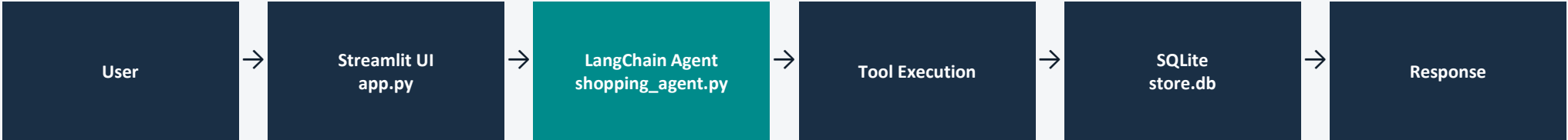
Chat-first interface feels like a real shopping assistant, not a form

Sidebar image upload keeps text and photo search in one flow

Product IDs and prices shown inline build trust before ordering

The same agent powers both the CLI entry point and the Streamlit app

Repository Structure & Request Flow



Project File Architecture

app.py	Streamlit web app — chat orchestration, sidebar image upload
shopping_agent.py	Agent setup, tool definitions, and system prompt rules
reviews_api.py	Rating aggregation against the reviews table
setup_db.py	Schema creation and seed data for products, reviews, orders
resources/	Sample images (honey, oats, elephant) for image-search demos
requirements.txt	Runtime dependencies — LangChain, Groq, Streamlit, and more

KEY TAKEAWAYS

What This Project Demonstrates

Agentic Tool-Calling Design

Explicit, auditable tools instead of one giant prompt show production-grade LLM application architecture.

Multimodal Product Discovery

Combining text and vision inputs into a single shopping flow demonstrates versatile LLM integration.

Transaction-Safe Conversational UX

Confirmation guardrails around checkout show real-world safety and auditability thinking.

Full-Stack AI Application

From the SQLite data layer to LangChain orchestration to a deployed Streamlit UI — an end-to-end, working assistant.

Sajal Agarwal

AI & Machine Learning Engineer

PROJECT REPOSITORY

github.com/Sajalagarwal-ca/shopping_agent

github.com/Sajalagarwal-ca/healthcare_premium_prediction

4

Agent Tools

2

Input Modes

3

DB Tables

2 LLMs

Groq Models

Built with Python · LangChain · Groq · Streamlit · SQLite

Portfolio Project — Conversational AI & Agentic Systems